Rapport de certification

Djé-Rhums : site vitrine de rhums arrangés réalisé avec Wagtail

Réaliser des applications web à l'aide d'un CMS · Découverte Métier IA, Simplon Bordeaux · Jérôme Delacoudre · Novembre 2026

Résumé exécutif

Jérôme, surnommé « Djé », est le créateur (fictif) de Djé-Rhums, une marque de rhums arrangés artisanaux installée à Cadaujac, près de Bordeaux. Il voulait un site vitrine pour faire connaître ses produits, recevoir des demandes de commande par un formulaire et gérer lui-même ses contenus. J'ai réalisé ce site avec le CMS Wagtail : un accueil, un catalogue de 13 produits en deux catégories, une page Événements avec ses partenaires, un formulaire de contact et de commande, et les pages légales. Djé peut ajouter ou modifier ses produits, ses événements et ses partenaires depuis l'administration. Ses employés ont des droits limités et ne peuvent pas publier seuls. Le site est pensé « mobile first » et obtient 100/100 en accessibilité et en référencement avec l'outil Lighthouse. [Mise en ligne : à compléter après le déploiement, avec l'adresse du site.]

Introduction : contexte du projet

Djé-Rhums est un projet commercial fictif, décrit dans un cahier des charges. Djé est passionné de rhum. Il prépare à la main des rhums arrangés, c'est-à-dire des rhums parfumés avec des fruits, aux saveurs inhabituelles (ananas-passion-vanille, fraise-myrtille, mangue-carotte…). Il veut les vendre au détail comme en gros, à des particuliers et à des professionnels (bars, restaurants), dans la région bordelaise.

Aujourd'hui, ses produits n'ont aucune présence en ligne. Un client curieux ne peut ni voir les bouteilles, ni comparer les saveurs, ni passer commande sans le contacter directement. Sa clientèle est faite de curieux et de novices, qui choisissent surtout avec les yeux, grâce aux photos. Comme il vend de l'alcool, il doit aussi respecter des règles : interdiction de vente aux mineurs et protection des données de ses clients. Enfin, Djé n'est pas développeur : il veut garder la main sur son catalogue sans dépendre de quelqu'un.

1. Analyse et compréhension du besoin C1

J'ai rédigé un cahier des charges (voir **Annexe 1**), dont je résume ici les points principaux.

- **Objectifs :** quatre résultats à obtenir : être trouvé sur Google dans la région bordelaise, recevoir des demandes de commande (particuliers et professionnels), présenter l'offre clairement, et être autonome pour mettre à jour le site.
- **Utilisateurs cibles :** ceux qui découvrent (novices, curieux), ceux qui commandent (clients, bars, restaurants), ceux qui administrent (Djé et ses employés, avec des droits limités).
- **Caractéristiques d'usage :** site « mobile first » (consulté d'abord sur téléphone), formulaire simple, une adresse stable par produit, photos légères, référencement local.
- **Contenus :** quatre pages principales (Accueil, Produits, Contact, Événements) et une fiche par produit.
- **Données personnelles :** le formulaire collecte nom, e-mail, téléphone, adresse et commande. J'ai prévu un consentement non pré-coché, une case « j'ai plus de 18 ans » (sans date de naissance, pour limiter les données), une page de politique de confidentialité et la mention légale sur l'alcool.

Comment le cadrage a évolué

Mon ordre de travail a été : cadrage, schéma fonctionnel, maquettes (Figma), projet Wagtail, accueil, produits, contact, événements, réglages et vérifications. En avançant, j'ai adapté le cadrage et j'ai mis à jour le cahier (version 2) :

- **Recettes :** le cahier prévoyait une recette et un mode de fabrication sur chaque fiche. Je les ai **supprimées** : ce sont les secrets de fabrication de Djé, qui ne veut pas les rendre publics.
- **Fiches produit :** prévues en contenu géré par un snippet, je les ai finalement faites en **pages Wagtail**, car une page a sa propre adresse et ses propres balises de référencement, ce qu'un snippet n'a pas.
- **Partenaires :** prévu comme un simple texte sur la page Événements, j'en ai fait un **snippet** réutilisable (nom, logo, lien).
- **Pages légales :** le cahier ne prévoyait que la confidentialité. J'ai **ajouté les mentions légales**, obligatoires pour un site commercial.

2. Préparer la solution technique

Schéma fonctionnel, choix du CMS et des extensions C2

Le schéma fonctionnel (**Annexe 2**) montre les pages et leurs liens. J'ai réalisé les maquettes dans Figma, pour le mobile et pour l'ordinateur (**Annexe 3**).

Pourquoi Wagtail

- Son **arbre de pages** correspond exactement à l'arborescence du cahier (Accueil, puis Produits, puis une fiche par produit) et donne des adresses propres.
- Ses **formulaires sont administrables** : Djé peut modifier les champs sans toucher au code.
- Il **redimensionne les images** automatiquement, utile pour des visiteurs qui découvrent les produits par la photo, sur mobile.
- Son **administration est simple** et gère des groupes avec des droits distincts, ce qu'il faut pour les employés.
- Chaque page a des champs pour le **titre et la description de référencement**.

Les extensions

Une extension est un module optionnel, une « app » qu'on active dans le réglage *INSTALLED_APPS*, pour ajouter une fonctionnalité au CMS. Voici celles que j'utilise :

- *wagtail.contrib.forms* : formulaire de contact et de commande dont les champs se gèrent dans l'admin, avec envoi par e-mail et enregistrement des demandes.
- *wagtail.contrib.redirects* : quand je change l'adresse d'une page, l'ancienne adresse redirige vers la nouvelle (utile quand j'ai nettoyé les adresses des produits).
- *wagtail.snippets* : éléments réutilisables sans adresse propre, pour les partenaires.
- *wagtail.images* : médiathèque et redimensionnement des photos de produits.

Les extensions *documents*, *embeds* et *search* sont présentes par défaut dans le projet, mais je ne m'en sers pas. L'extension de plan du site (*sitemaps*) ainsi qu'un fichier *robots.txt* ont aussi été ajoutés, pour le volet référencement (voir Annexe 4).

Préparation de l'environnement de travail C3

J'ai préparé l'environnement en suivant [à confirmer : la documentation officielle de Wagtail « Your first Wagtail site » / le brief de la formation]. Voici les commandes, avec leur rôle :

```
python -m venv env
```

Crée un environnement virtuel (dossier *env*) : les paquets du projet sont isolés du reste de l'ordinateur.

```
env\Scripts\activate
```

Active cet environnement (sous Windows).

```
pip install wagtail
```

Installe Wagtail 8.1, avec Django 6.1.3 (Python 3.15). Les versions exactes sont listées dans *requirements.txt*.

```
wagtail start djerhums
```

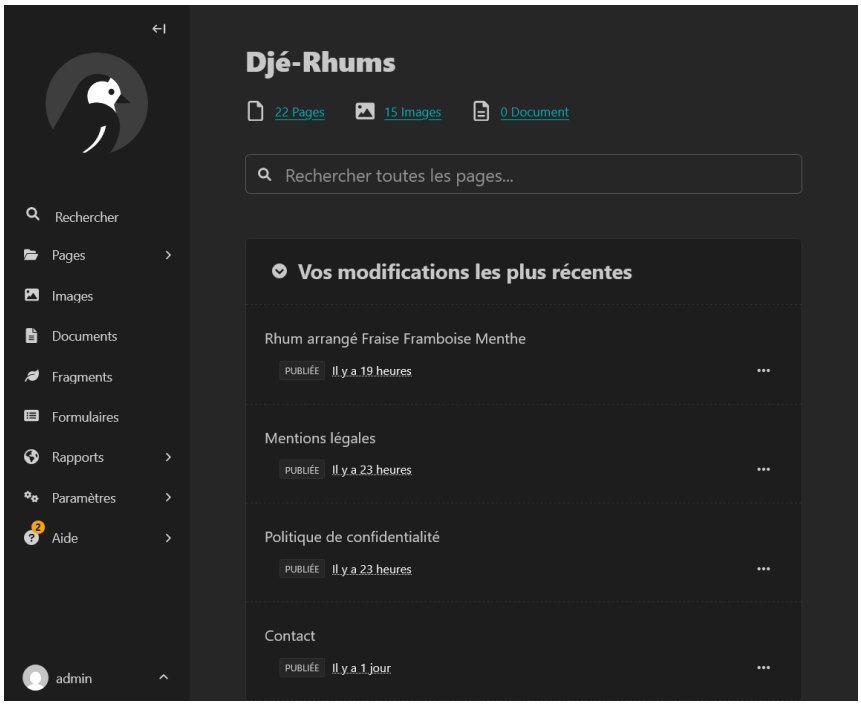
Génère le projet : réglages séparés (*base*, *dev*, *production*), applications *home* et *search* déjà créées.

```
python manage.py startapp produits
```

Crée une nouvelle application (idem pour *evenements* et *contact*), une par partie du site.

```
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Crée les tables de la base de données, crée le compte administrateur, puis lance le serveur de développement pour vérifier que tout fonctionne.



Le tableau de bord de l'administration : 22 pages, 15 images, 0 document, et le menu Fragments pour les partenaires.

Tip technique : après chaque étape importante, je lance *python manage.py check* (détecte les erreurs de configuration) et *python manage.py makemigrations --check* (vérifie qu'aucun changement de modèle n'a été oublié).

3. Réalisation de l'application

Explication des choix de paramétrage C4

J'ai découpé le site en quatre applications Django : *home*, *produits*, *evenements* et *contact*, toutes déclarées dans *INSTALLED_APPS*.

La hiérarchie des pages

Avec *parent_page_types* et *subpage_types*, je limite l'endroit où chaque page peut être créée. Une fiche produit ne peut exister que sous la page « Nos rhums arrangés » : Djé ne peut pas la ranger au mauvais endroit.

```
class ProduitPage(Page):
    parent_page_types = ["produits.ProduitsIndexPage"]
    subpage_types = []
```

Le formulaire de contact et de commande

La page *ContactPage* hérite de *AbstractEmailForm* (extension *forms*). Un panneau *InlinePanel* permet à Djé d'ajouter ou de retirer des champs dans l'admin ; chaque champ est un objet *FormField* relié à la page par une *ParentalKey*. L'adresse d'envoi et le sujet se règlent aussi dans l'admin. En développement, les e-mails s'affichent dans le terminal (réglage *MAILERS* en mode console) ; l'envoi réel sera configuré au déploiement.

```
class ContactPage(AbstractEmailForm):
    intro = RichTextField(blank=True)
    content_panels = AbstractEmailForm.content_panels + [
        FieldPanel("intro"),
        InlinePanel("form_fields", label="Champs du formulaire"),
    ]
```

Dans le gabarit, la balise *{% csrf_token %}* est obligatoire sur tout formulaire qui envoie des données (méthode POST) : elle ajoute un jeton caché qui empêche un autre site d'envoyer le formulaire à ma place.

```
<form action="{% pageurl page %}" method="post" class="formulaire-contact">
    {% csrf_token %}
```

Deux cases à cocher sont obligatoires : « j'ai plus de 18 ans » et le consentement RGPD. Un petit script JavaScript affiche le bloc « Produits commandés / Quantité » seulement quand le motif choisi est « Commande », en s'appuyant sur l'attribut *hidden* et une règle CSS :

```
[hidden] { display: none !important; }
```

Le snippet Partenaire

Un partenaire n'a besoin ni d'adresse propre ni de référencement : c'est une simple fiche réutilisable (nom, logo, lien). J'en ai fait un **snippet** avec le décorateur *@register_snippet*. Dans l'administration en français, ce menu s'appelle « Fragments ».

```
@register_snippet
class Partenaire(models.Model):
    nom = models.CharField(max_length=100)
    logo = models.ForeignKey("wagtailimages.Image", null=True, blank=True,
on_delete=models.SET_NULL, related_name="+")
    lien = models.URLField(blank=True)
    panels = [FieldPanel("nom"), FieldPanel("logo"), FieldPanel("lien")]
```

Les droits des employés

J'ai créé un groupe **Employés**, avec des droits limités :

- **Autorisé :** accéder à l'administration ; ajouter et modifier les pages sous « Nos rhums arrangés » et « Événements » ; ajouter et modifier les partenaires ; ajouter, modifier et choisir des images.
- **Interdit :** publier une page (ses changements passent par une validation de Djé) ; supprimer ; modifier l'accueil, le contact et les pages légales ; gérer les utilisateurs et les groupes ; lire les demandes reçues.

J'ai testé avec un compte d'essai (supprimé ensuite) : le bouton « Publier » n'apparaît pas, il reste « Soumettre à la modération », et les pages Accueil et Contact ne sont pas modifiables.

Les redirections et les réglages

En remplaçant les adresses contenant des accents par des adresses propres (*WAGTAIL_ALLOW_UNICODE_SLUGS = False*), l'extension *redirects* a créé les redirections automatiquement. J'ai aussi réglé la langue et le fuseau horaire du site :

```
LANGUAGE_CODE = "fr-fr"
TIME_ZONE = "Europe/Paris"
```

Création et réalisation des interfaces utilisateurs C5

Gabarits (templates). Le fichier *base.html* contient ce qui est commun à toutes les pages : le lien d'évitement, l'en-tête avec le logo et le menu, le pied de page (réseaux sociaux, mentions légales, confidentialité, mention sur l'alcool). Chaque page fait un seul appel pour hériter de ce gabarit commun et ne remplit que son propre contenu :

```
{% extends "base.html" %}
```

Le nom du fichier suit le nom de la classe : *ProduitPage* donne *produit_page.html*, et Wagtail le trouve tout seul, sans configuration supplémentaire. La carte produit est écrite une seule fois (*carte_produit.html*) et réutilisée sur l'accueil et sur la liste, avec *{% include %}*.

Personnalisation graphique. J'ai défini ma palette dans des variables CSS, réutilisées partout sur le site :

```
:root {
    --color-primary: #8B3A2B;
    --color-secondary: #C86D3F;
    --color-accent: #E9C46A;
}
```

Les couleurs de texte respectent le contraste du niveau AA. La mise en page est « mobile first » : le CSS de base est écrit pour le téléphone, puis une seule règle adapte la tablette et l'ordinateur à partir de 768 pixels de large.

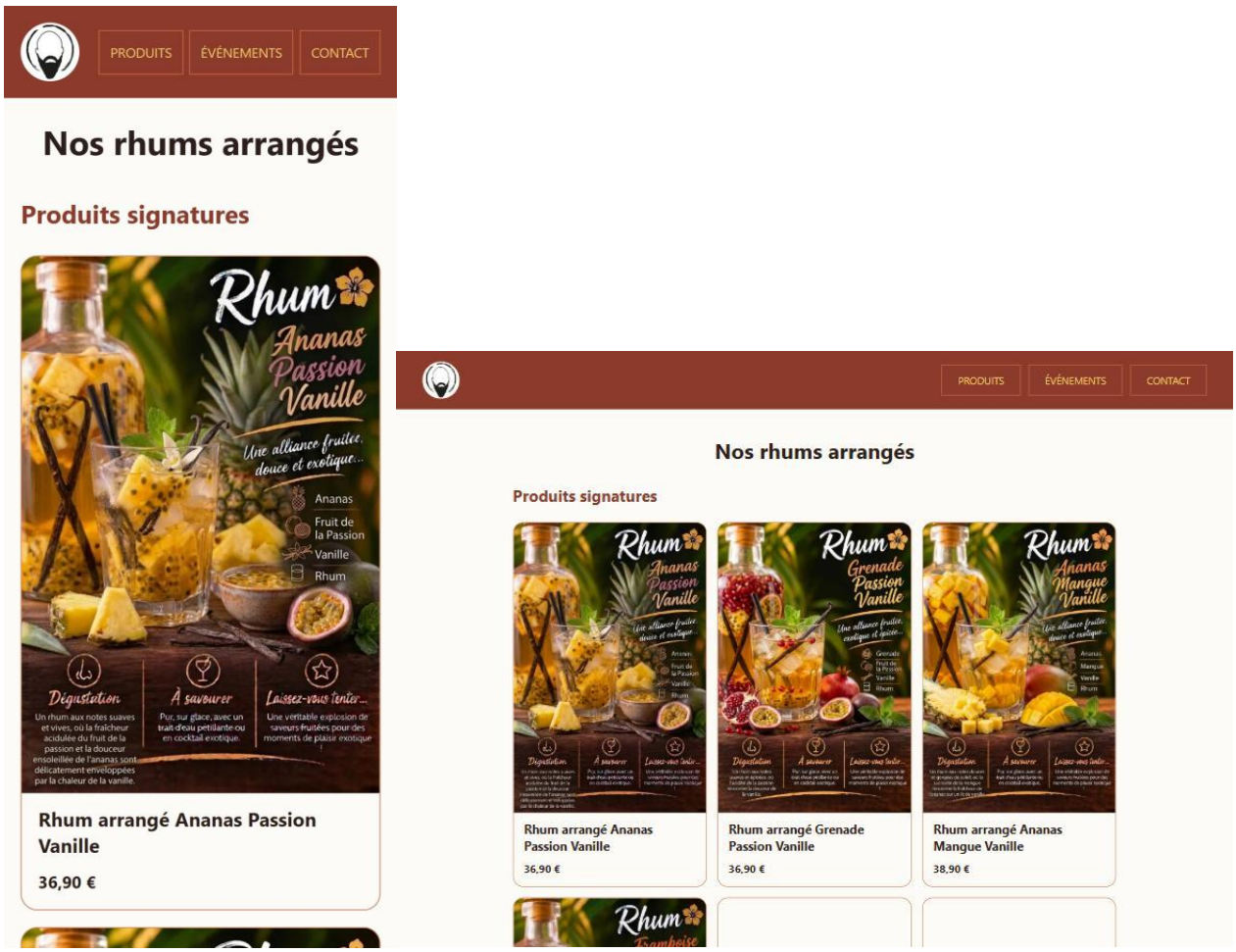
```
.grille-produits { display: grid; grid-template-columns: 1fr; gap: 16px; }
@media (min-width: 768px) {
    .grille-produits { grid-template-columns: repeat(3, 1fr); }
}
```

Du wireframe à la réalisation

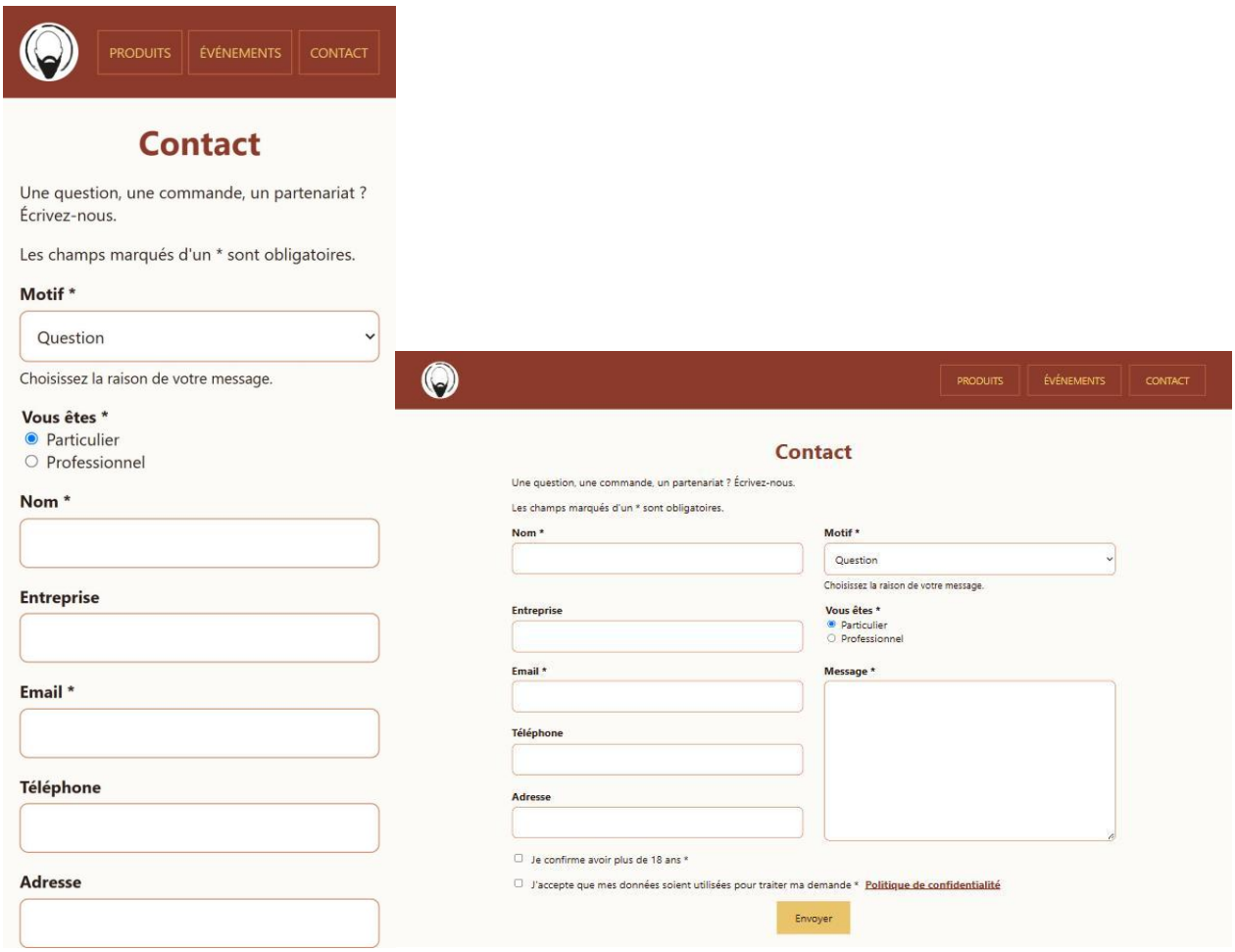
- **Accueil :** même structure que le wireframe (titre, image et présentation, produits, créateur). J'ai ajouté le prix sur les cartes et le titre « Mon histoire ».
- **Nos rhums arrangés :** deux sections (produits signatures, créations audacieuses). Les wireframes prévoyaient 2 colonnes sur mobile : j'ai gardé **1 colonne**, car en réduisant les photos de moitié, les mises en scène perdent leur effet.
- **Fiche produit :** image, descriptif, prix, degré d'alcool, retour. Plus de bloc « recette » (voir la décision en partie 1).
- **Événements :** cartes avec la date, liste des partenaires, liens vers les réseaux sociaux.
- **Contact :** formulaire en 1 colonne sur mobile, en 2 colonnes sur ordinateur, comme sur le wireframe.



Accueil, mobile puis ordinateur (haut de page) — à comparer avec l'Annexe 3, pages 2 et 10.



Nos rhums arrangés, mobile puis ordinateur — à comparer avec l'Annexe 3, pages 3 et 11.



Contact, mobile puis ordinateur — à comparer avec l'Annexe 3, pages 7 et 15.

> Tip technique : quand le CSS « ne bouge pas » après une modification, je fais Ctrl+Maj+R : le navigateur garde l'ancienne feuille de style en mémoire.

Pour que le clavier reste utilisable, j'ai ajouté un contour visible qui entoure toute la carte quand on la sélectionne au clavier, grâce au sélecteur CSS *:has()* :

```
.carte-produit:has(a:focus-visible) {
    outline: 3px solid var(--color-primary);
}
```

Intégration des contenus et hiérarchisation C6

Les pages ont été créées dans l'admin en suivant l'arborescence du cahier :

```
Accueil
  +-- Nos rhums arrangés (13 fiches produit)
  +-- Contact
  +-- Événements (3 fiches événement)
  +-- Politique de confidentialité
  +-- Mentions légales
```

Les produits. Chaque fiche a une photo, une description, un prix, un degré d'alcool, une catégorie et une case « Afficher sur l'accueil » : la page d'accueil montre automatiquement les produits cochés, et la liste les range selon leur catégorie.

Les images. J'ai importé 13 photos de produits dans la médiathèque (plus le logo et l'image d'accueil). Le gabarit demande à Wagtail une version adaptée de chaque photo :

```
{% image produit.image fill-600x900 format-webp loading="lazy" alt=produit.title %}
```

Cette ligne redimensionne la photo, la convertit au format WebP et ne la charge que lorsque le visiteur approche. Sur une photo, j'ai mesuré **166 Ko en JPEG contre 92 Ko en WebP**, soit environ 45 % de moins.

Pour le texte riche saisi dans l'admin (description, introduction), le filtre *richtext* transforme le contenu enregistré en HTML affichable :

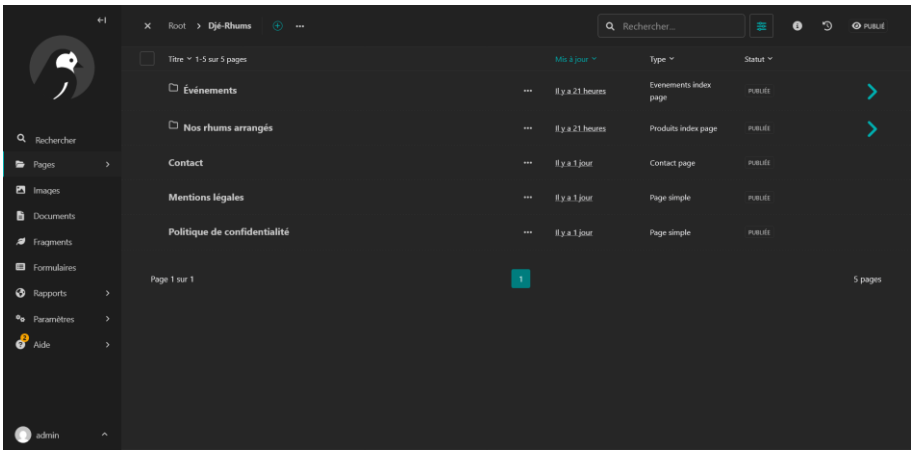
```
{{ page.description|richtext }}
```

La hiérarchie. Chaque page a un seul titre de niveau 1 et des titres de niveau 2 pour ses sections. La langue de la page est déclarée une fois pour toutes dans le gabarit commun :

```
<html lang="fr">
```

J'ai aussi renseigné un titre et une description de référencement sur chaque page, avec « Cadaujac », « Bordeaux » ou « Gironde ». Un contrôle automatique des 22 pages a confirmé : un seul *h1* par page, aucune image sans texte alternatif, descriptions de 70 à 160 caractères.

Remarque : les contenus (photos d'événements, partenaires, liens des réseaux) sont des exemples : l'objectif du projet est la construction du site.



L'arbre des pages dans l'admin, sous Djé-Rhums.

CAPTURE À INSÉRER : l'écran de saisie d'une fiche produit (ouvrir une page "Rhum arrangé..." en édition, pas la liste).

Problèmes rencontrés

Problème 1 : une page d'erreur Django jaune

Après avoir ajouté un texte d'aide dans le formulaire, la page s'est remplacée par un écran jaune : « *TemplateSyntaxError : Invalid block tag on line 48: 'endblock', expected 'empty' or 'endfor'* ».

Problème 2 : un doute sur une migration

J'ai lancé *python manage.py makemigrations* sans préciser l'application, juste après avoir créé les modèles du formulaire, et je me suis demandé si c'était un problème.

Problème 3 : des photos coupées en haut

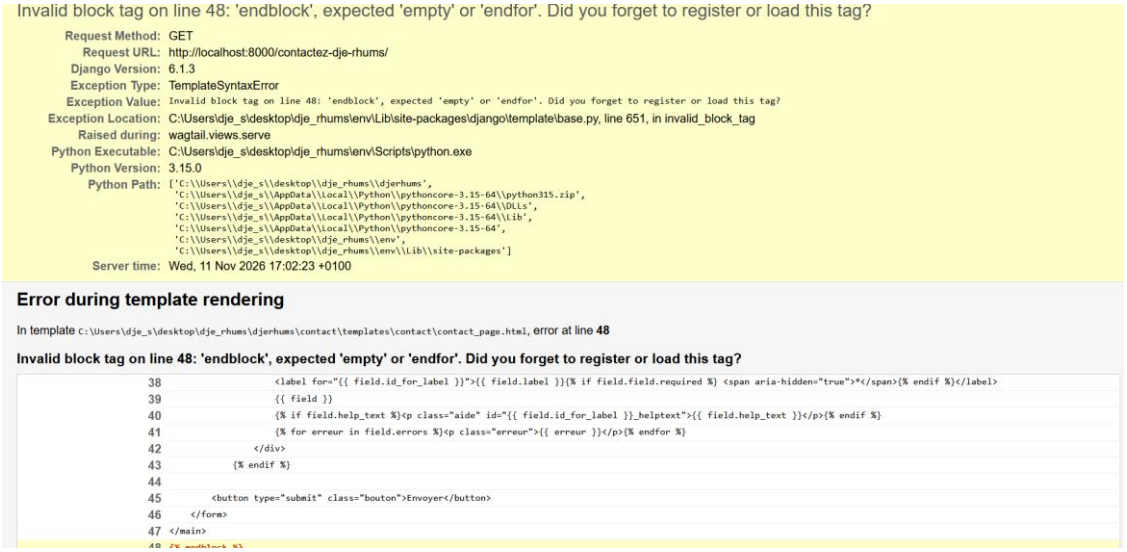
Sur les cartes produit, le haut des étiquettes (le nom du rhum) était rogné, alors que mes photos étaient correctes.

Solutions apportées

Solution 1 : retrouver la balise oubliée

Délimiter le problème. Le message dit ce qu'il attend : un *endfor*. Une boucle *{% for %}* était donc restée ouverte dans le fichier *contact_page.html*. Le problème est apparu juste après ma dernière modification, donc c'est là qu'il fallait chercher. **Chercher.** J'ai compté les balises : 4 *{% for %}* ouvrants pour seulement 3 *{% endfor %}*. **Solution.** En ajoutant ma ligne, j'avais supprimé par erreur le *{% endfor %}* de la grande boucle des champs. Je l'ai remis juste après le dernier *{% endif %}*, et la page s'est réaffichée.

Tip technique : face à une *TemplateSyntaxError*, je lis ce que Django attend, puis je compare le nombre de balises ouvrantes et fermantes (*if*, *for*, *block*).



La page d'erreur Django, avec la balise manquante repérée ligne 48.

Solution 2 : vérifier au lieu de supposer

Délimiter. La question était : qu'a réellement créé la commande ? **Chercher.** J'ai relu la sortie du terminal, listé les migrations avec *python manage.py showmigrations* et lancé *makemigrations --check*. **Résultat.** Il n'y avait pas d'erreur : sans nom d'application, Django regarde toutes les applications et crée une migration pour chacune qui a changé. Seule *contact* avait des changements, donc le résultat était identique à *makemigrations contact*. **Ce que j'en retiens.** Je lis toujours la sortie de la commande avant de lancer *migrate*, et je vérifie ensuite avec *--check* (réponse attendue : « No changes detected »).

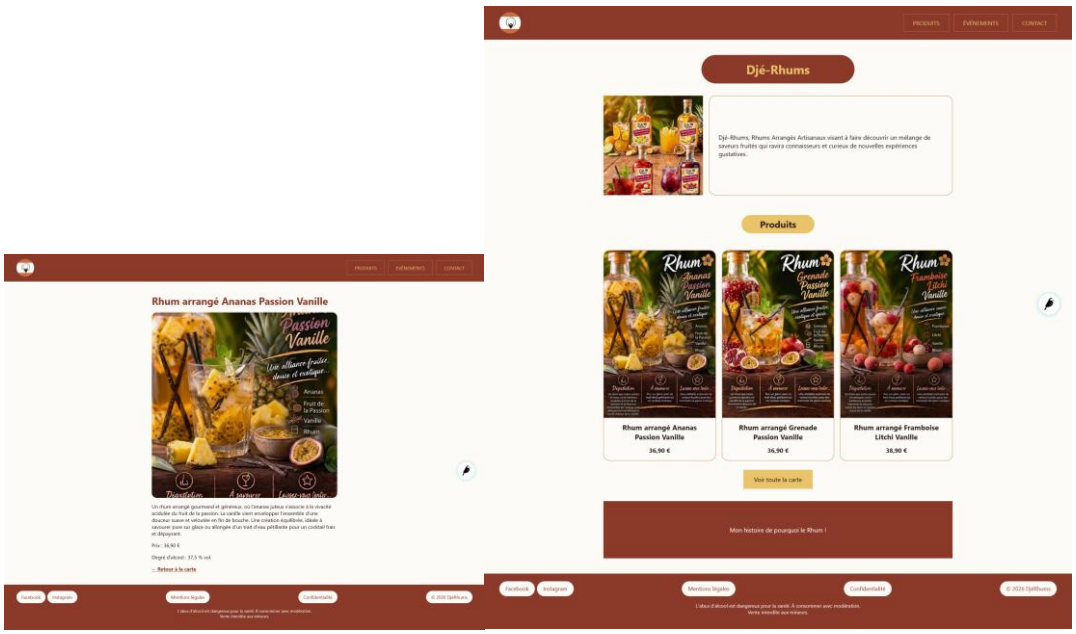
Solution 3 : demander à Wagtail le bon format d'image

Délimiter. Seules les images étaient touchées, pas le CSS. Mes 13 photos font 843 × 1264 pixels : un format portrait 2:3. Dans mon gabarit, je demandais un format carré. **Chercher.** Dans la documentation, le mot-clé *fill* veut dire « remplir exactement ce format en rognant ce qui dépasse » : Wagtail coupait donc environ un tiers de la hauteur. **Solution.** Demander un format de même proportion :

```
{# avant : format carré, qui rogne le haut et le bas #}
{% image produit.image fill-400x400 alt=produit.title %}

{# après : même proportion que la photo (2:3), rien n'est rogné #}
{% image produit.image fill-400x600 alt=produit.title %}
```

Comme la carte est un fichier unique, la correction s'est appliquée d'un coup à l'accueil et à la liste. J'ai ensuite ajouté *width: 100%* en CSS (pour supprimer un espace blanc dans les cartes en 1 colonne) et agrandi la version générée à 600 × 900.



Avant (étiquette rognée en haut) et après (cartes avec l'étiquette entière).

4. Déploiement de l'application C8

[Partie à compléter après le déploiement : la solution d'hébergement choisie et sa justification, la configuration du nom de domaine (Djé-Rhums.com), l'outil d'analyse de trafic et la stratégie de mise à jour du CMS. Réglages de production à traiter : adresse du site dans Wagtail, clé secrète, mode débogage désactivé, HTTPS, envoi réel des e-mails.]

Tests déjà réalisés en local

- *python manage.py check* et *makemigrations --check* sans erreur.
- Contrôle automatique des 22 pages (code 200, un seul titre de niveau 1, descriptions, images avec texte alternatif, liens internes valides).
- Formulaire testé : refusé si la case « +18 ans » n'est pas cochée, accepté sinon, e-mail visible dans le terminal.
- Compte Employés testé : pas de bouton « Publier », accès refusé à l'accueil et au contact.
- Lighthouse : 100/100 en accessibilité et en référencement sur les pages testées.

5. Bilan et conclusion

Bilan du projet

Le site répond aux quatre objectifs du cahier des charges. **Être trouvé localement :** atteint côté site (titres et descriptions citant Cadaujac, Bordeaux et la Gironde), même si l'indexation réelle sur Google dépendra de la mise en ligne. **Recevoir des demandes :** atteint, avec un formulaire fonctionnel qui vérifie la majorité et le consentement. **Présenter l'offre clairement :** atteint, avec 13 produits rangés en deux catégories et des fiches simples. **Être autonome :** atteint, les produits, les événements et les partenaires s'administrent sans développeur, avec des droits limités pour les employés.

Points perfectibles : les contenus sont des exemples, et l'envoi réel des e-mails n'est pas configuré. Ma mesure de performance Lighthouse a échoué en local (erreur de l'outil, pas du site) : une première mesure donnait 81/100, avant l'allègement des images, et sera à refaire en ligne.

Améliorations envisagées

- Afficher des avis de clients sur les fiches produit (prévu dans le schéma fonctionnel).
- Configurer l'envoi d'e-mails réel et une notification pour Djé.

- Remplacer les exemples par les vrais partenaires, photos et réseaux sociaux.

Conclusion : apprentissages et perspectives

[Proposition à réécrire avec tes propres mots :] Ce projet m'a permis de comprendre concrètement comment Wagtail organise un site : des pages en arbre, des modèles qui décrivent les champs, des gabarits qui les affichent. Mes erreurs (une balise oubliée, un mauvais format d'image) m'ont appris à lire les messages d'erreur, à délimiter le problème et à vérifier avant d'avancer. J'ai aussi compris l'importance de l'accessibilité et du référencement dès la conception. [Ajouter : ce que ça change pour ton projet professionnel.]

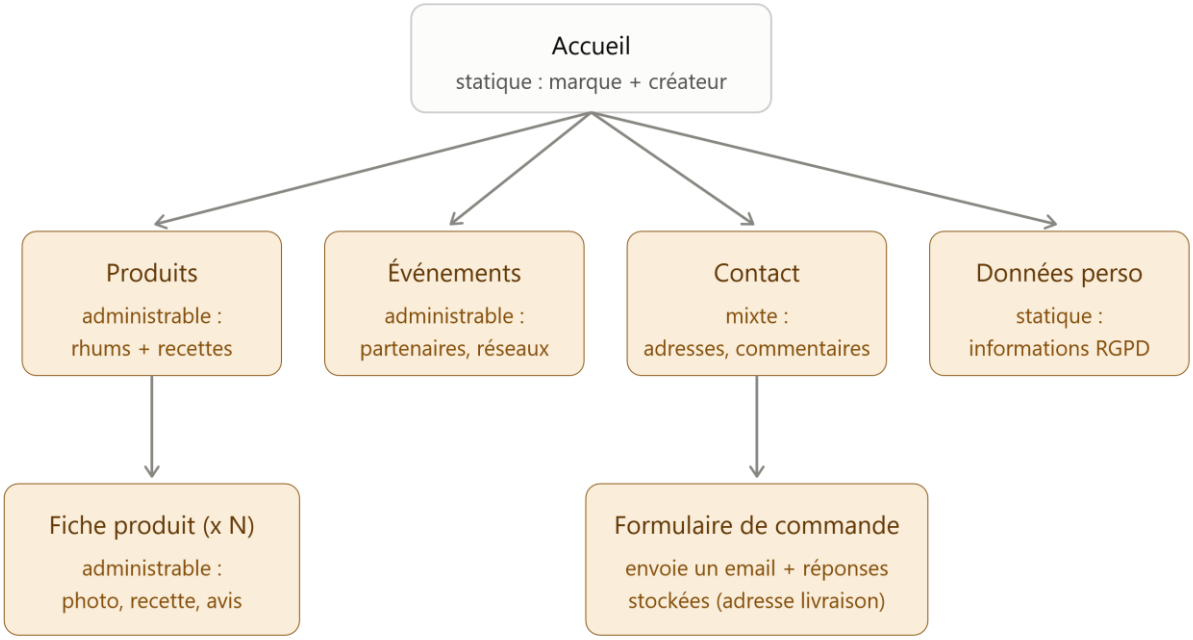
Annexes

Annexe 1 : Contenu du cahier des charges (version 2)

Le document complet (7 pages) est fourni à part. En voici le sommaire détaillé :

- **0. Présentation du projet :** Djé-Rhums, projet fictif de rhums arrangés à Cadaujac (Gironde), vente au détail et en gros, site vitrine sans panier ni paiement en ligne.
- **1. Objectifs :** être trouvé localement sur Google, recevoir des demandes de commande, présenter l'offre clairement, être autonome sur le contenu.
- **2. Utilisateurs cibles :** découvrir (novices/curieux), commander (particuliers, bars, restaurateurs), administrer (Djé, puis employés avec des droits limités).
- **3. Caractéristiques d'usage :** mobile first, formulaire en une colonne sur mobile, URL stable par produit, images optimisées, référencement local, accessibilité (lien d'évitement, clavier, contrastes AA).
- **4. Contenus et données :** arborescence des pages (Accueil, Nos rhums arrangés + fiches, Contact, Événements + fiches, Confidentialité, Mentions légales ; Partenaire en snippet) ; détail du formulaire de contact/commande (motif, qualité, nom, email, téléphone, adresse, produits/quantité, message, case +18 ans, consentement) ; contenu géré par l'administrateur ; rôles et droits (Administrateur / groupe Employés).
- **5. Nom de domaine :** Djé-Rhums.com, disponibilité vérifiée, 12 $/an.
- **6. Enjeux liés aux données personnelles :** données collectées (aucune sensible), information, consentement, finalité, durée de conservation (3 ans / 10 ans pour les commandes, à confirmer auprès de la CNIL), droits des personnes, sécurité, mention alcool.
- **7. Éléments restant à compléter avant la mise en ligne :** mentions légales (SIRET, hébergeur), nom de l'hébergeur pour la confidentialité, vrais partenaires/photos/réseaux sociaux, registrar du nom de domaine, HTTPS et emails en production.
- **8. Historique des modifications (version 2) :** tableau détaillé des écarts avec la version initiale (recettes supprimées, fiches produit en pages, partenaires en snippet, pages légales ajoutées, formulaire précisé, droits employés précisés) — repris en partie 1 de ce rapport.

Annexe 2 : Schéma fonctionnel



Annexe 3 : Wireframes (version 2)

Le document complet (17 pages, Figma) liste les 8 écrans du site en version mobile puis en version ordinateur. Capture du tableau des changements depuis la v1, puis un exemple d'écran (Accueil, mobile) :

Wireframes – DjéRhums

Version 2 · 13 novembre 2026 · mise à jour d'après le site réalisé

Pages (mobile puis ordinateur)	Changements depuis la version 1
Accueil	Cartes avec nom et prix, bloc « Mon histoire », 1 colonne sur mobile
Nos rhums arrangés	Sections « Produits signatures » et « Créations audacieuses » (au lieu de « Carte saisonnière »), cartes verticales avec prix
Fiche produit	Plus de recette ni de bouton « Commander » ; prix et degré d'alcool ajoutés
Événements	Introduction, date sur les cartes, partenaires (snippets), section « Suivez-nous »
Fiche événement	**Nouvelle page** : date, lieu, photo, description
Contact	Introduction, champs obligatoires (*), case +18 ans, consentement avec lien, phrase RGPD
Confirmation d'envoi	**Nouvelle page**
Pages légales	**Nouvelles pages** : mentions légales et politique de confidentialité
Toutes les pages	Menu « Événements », footer complet (réseaux, mentions, confidentialité, ligne locale, mention alcool)

Légende

Image emplacement d'une image Texte bloc de texte Bouton bouton

Accueil : mobile

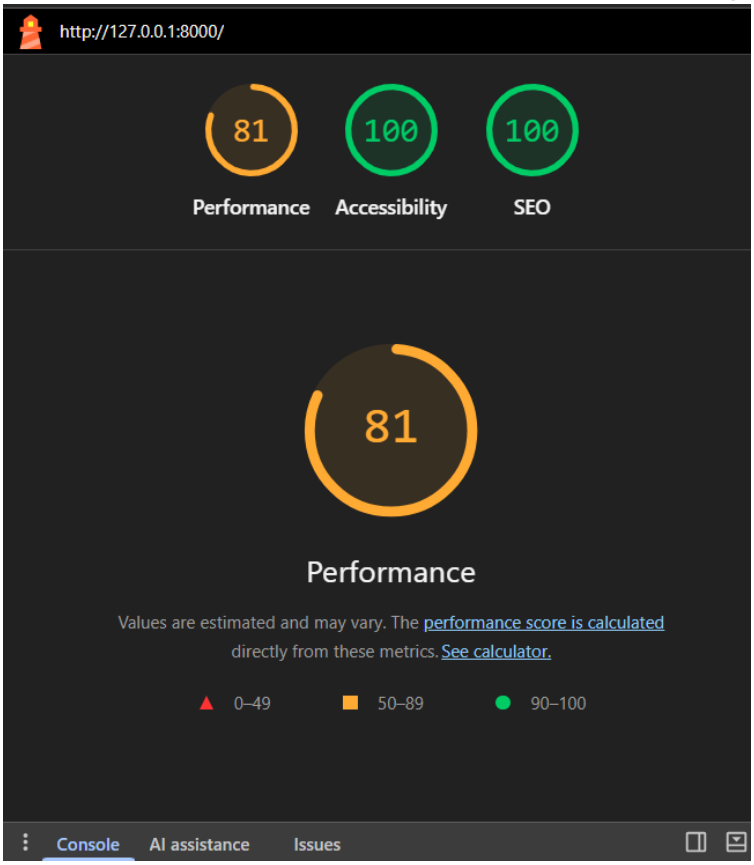
Largeur 375 px



Annexe 4 : Référencement et accessibilité

Cette annexe résume ce que j'ai mis en place pour le référencement et l'accessibilité, avec des vérifications que j'ai faites.

- **Titres et descriptions :** renseignés sur les 22 pages, descriptions de 70 à 160 caractères, avec la zone locale (Cadaujac, Bordeaux, Gironde).
- **Adresses (URL) :** lisibles et sans accents, redirections automatiques quand une adresse change.
- **Structure :** langue de la page déclarée (*lang="fr"*), un seul titre de niveau 1 par page, titres de niveau 2 pour les sections.
- **Images :** texte alternatif sur toutes les images, version redimensionnée en WebP, chargement différé.
- **Navigation au clavier :** lien « Aller au contenu principal » en tout début de page ; contour visible sur les liens, les boutons et les cartes (*:focus-visible*).
- **Contrastes :** couleurs choisies pour respecter le niveau AA (rapport de contraste de 4,5:1 pour le texte).
- **Formulaire :** chaque champ a son *label* ; les boutons radio sont dans un *fieldset* avec *legend* ; astérisque et phrase « champs obligatoires » ; message d'erreur global (*role="alert"*) et erreur sous chaque champ ; attributs *autocomplete* pour aider la saisie.
- **Liens :** menu avec *aria-label* ; liens qui s'ouvrent dans un nouvel onglet annoncés par un *aria-label*.
- **Résultats Lighthouse :** 100/100 en accessibilité et en référencement sur les pages testées (capture ci-dessous). Une mesure de performance antérieure à l'allègement des images donnait 81/100 ; une mesure plus récente a échoué en local (erreur *NO_NAVSTART* de l'outil, pas du site).
- **Plan du site et robots.txt :** *sitemap.xml* (généré automatiquement par Wagtail, une entrée par page publiée) et *robots.txt* (bloque l'admin, pointe vers le sitemap), ajoutés pour le cas pratique référencement.
- **Reste à faire :** renseigner le vrai nom de domaine dans les paramètres du site (Wagtail), aujourd'hui le sitemap utilise « localhost » ; mesure de performance en ligne.



Rapport Lighthouse : 81/100 en performance (avant l'allègement des images), 100/100 en accessibilité et en référencement.